Universidade Federal de Uberlândia
Graduação em Gestão da Informação: Bacharelado - Integral
FACOM39803: Mineração de Dados – Prof. Dr. Murillo G. Carneiro

Mineração de Dados Aplicada à Detecção de Fraudes em Pagamentos Online

Adryell Alexandre Campos Medeiros - 12221GIN012
Guilherme Costa Rodrigues – 12111GIN018
Lucas Ramos Fernandes da Silva - 12211GIN010
Matheus Ribeiro Prado – 12211GIN016

Sumário

Resumo **2**
1. Introdução **2**
1.1 Definição do problema 2
1.2 Tarefa de mineração e técnicas escolhidas 2
2. Desenvolvimento **3**
2.1 Pré-processamento **3**
2.2 Mineração de dados (treinamento) **4**
2.2.1 Seleção de Algoritmos e Configuração 4
2.2.2 Estratégia de Mitigação do Desbalanceamento 4
2.2.3 Validação Cruzada e Estabilidade 4
2.2.4 Persistência dos Modelos 5
2.3 Pós-processamento (avaliação e figuras) **5**
3. Resultados **5**
Referências **13**

Resumo

Este trabalho aborda a detecção de fraudes em transações financeiras digitais por meio de mineração de dados utilizando a base pública Online Payment Fraud Detection do Kaggle. O pré-processamento abrange a limpeza dos dados, a codificação one-hot, a conversão de tipos numéricos para float32 visando a redução do consumo de memória e a divisão estratificada entre conjuntos de treino e teste. A avaliação ocorre via validação cruzada com 3 folds. Dois classificadores são treinados: Random Forest e Regressão Logística, configurados com pesos de classe balanceados e avaliados pela métrica F1-macro. O pós-processamento gera matrizes de confusão para treino e teste, curvas ROC e gráficos de importância de variáveis. O código foi desenvolvido em Python, com as bibliotecas pandas, scikit-learn, matplotlib e seaborn, para execução em hardware limitado a 32 GB de RAM. Os resultados demonstram o desempenho dos algoritmos com base no F1-macro, precisão e recall.

1. Introdução

1.1 Definição do problema

A detecção de atividades fraudulentas em sistemas de pagamentos online constitui um desafio financeiro atual. As operações ilícitas causam perdas monetárias expressivas e diminuem a confiança dos usuários nas plataformas digitais. O objetivo deste projeto é desenvolver um sistema preditivo capaz de classificar transações financeiras como lícitas ou fraudulentas. Para realizar essa categorização, o modelo avalia atributos transacionais específicos, incluindo o tipo de operação realizada, os valores monetários transferidos e os saldos exatos das contas de origem e destino, tanto antes quanto depois da movimentação. A identificação automática e precisa desses padrões possibilita a emissão de alertas precoces e atua diretamente na contenção de perdas financeiras pelas instituições.

1.2 Tarefa de mineração e técnicas escolhidas

A tarefa estabelecida para a resolução do problema é a classificação supervisionada binária, tendo a variável alvo dividida entre as categorias fraude e não fraude. A metodologia adotada para o desenvolvimento do projeto estrutura-se de forma sequencial em três etapas principais: pré-processamento, mineração de dados e pós-processamento.

O pré-processamento efetua o carregamento da base de dados a partir de um arquivo CSV, a remoção de colunas com alta cardinalidade (como os identificadores únicos de usuários), a codificação da variável categórica referente ao tipo de transação por meio da técnica de one-hot encoding e a redução do tamanho alocado para os dados numéricos, convertendo-os para o formato float32. A base de dados completa é então particionada de forma estratificada, destinando 80% dos registros para o treinamento do modelo e 20% exclusivamente para a etapa de testes.

A etapa de mineração de dados treina os algoritmos Random Forest e Regressão Logística. O processo utiliza a validação cruzada com 3 folds e emprega o F1-macro como métrica principal de avaliação. O desbalanceamento inerente à base de dados de fraudes é tratado mediante o ajuste do parâmetro de peso das classes, definido como balanceado na configuração interna de ambos os modelos matemáticos. O classificador Random Forest

opera com parâmetros restritivos de proporção de amostras por árvore (max_samples) e concorrência de processos (n_jobs) rigorosamente ajustados para viabilizar a execução integral do código em um ambiente de hardware limitado a 32 GB de RAM.

O pós-processamento aplica os modelos treinados aos conjuntos de treino e teste para gerar as matrizes de confusão, os valores quantitativos de F1-macro e os relatórios de classificação. Curvas ROC e análises gráficas de importância das variáveis complementam as saídas visuais geradas pelo sistema.

2. Desenvolvimento

2.1 Pré-processamento

Para criar um modelo de aprendizagem supervisionada, etapas rigorosas de tratamento devem ser cumpridas. O código desenvolvido para esta fase encontra-se no arquivo preprocess.py. O script executa a leitura da base de dados e realiza a limpeza, removendo atributos que não agregam valor preditivo ao modelo, como as colunas nameOrig, nameDest e isFlaggedFraud.

Os algoritmos de machine learning exigem entradas numéricas. Como a base possui uma variável categórica para o tipo de transação, aplicou-se a função de one-hot encoding na coluna type. Para a divisão da base de dados, estabeleceu-se uma semente aleatória (seed) com valor 42, garantindo a reprodutibilidade da separação dos dados a cada execução do código. Utilizou-se o test_size de 20%, alocando 80% da base para treino. Para obter uma medida rigorosa na avaliação, aplicou-se a técnica stratify, que assegura a mesma proporção de itens fraudulentos nas bases de treino e de teste.

Após a finalização do tratamento, quatro arquivos CSV são gerados e armazenados: processed_X_train.csv (atributos de treino), processed_y_train.csv (rótulos de treino), processed_X_test.csv (atributos de teste) e processed_y_test.csv (rótulos de teste).

2.2 Mineração de dados (treinamento)

O processo de extração de padrões e ajuste dos modelos preditivos foi centralizado no módulo train.py. Esta etapa equilibra a exigência de alta sensibilidade na detecção de fraudes com as limitações computacionais do ambiente de execução.

2.2.1 Seleção de Algoritmos e Configuração

Foram implementados dois classificadores de naturezas distintas para possibilitar uma análise comparativa de desempenho:

- **Random Forest (Floresta Aleatória):** Algoritmo de aprendizagem baseada em comitês (Ensemble Learning). Foi configurado com 100 árvores de decisão (n_estimators=100). Para viabilizar a execução no sistema com 32 GB de RAM, utilizou-se o parâmetro max_samples=0.25, limitando cada árvore a processar apenas 25% do conjunto de treino. Esta técnica de subsampling reduz o pico de uso de memória sem comprometer a diversidade do modelo.

- **Regressão Logística:** Utilizada como modelo de base (baseline) devido à sua interpretabilidade e eficiência linear. Foi configurada com um limite de 1.000 iterações para garantir a convergência dos coeficientes na base de dados de grande escala.

2.2.2 Estratégia de Mitigação do Desbalanceamento

A base de dados apresenta uma desproporção severa entre transações lícitas e fraudulentas. Ambos os modelos foram instanciados com o parâmetro class_weight='balanced'. Esta configuração ajusta os pesos de maneira inversamente proporcional às frequências das classes nos dados de entrada, penalizando com maior rigor os erros cometidos na classificação da classe minoritária (fraude).

2.2.3 Validação Cruzada e Estabilidade

Para assegurar a estabilidade dos resultados e evitar viés em uma divisão específica dos dados, aplicou-se a Validação Cruzada de 3 Folds (k=3). O conjunto de treino foi subdividido em três partes, com o modelo sendo treinado em duas e validado na terceira de forma rotativa. As métricas monitoradas durante a validação cruzada foram:

- **F1-Macro:** Métrica principal, utilizada por equilibrar precisão e recall em ambas as classes de forma independente.

- **Recall (Revogação):** Medida necessária para aferir a capacidade do modelo em não deixar passar transações fraudulentas.

- **Precisão:** Medida importante para registrar o impacto de falsos bloqueios em clientes legítimos.

2.2.4 Persistência dos Modelos

Após a validação cruzada e o treino final com a base de treino completa, os objetos dos modelos foram serializados e gravados no disco rígido no formato .pkl através da biblioteca joblib. O procedimento permite o carregamento instantâneo do modelo para inferência ou pós-processamento, anulando a necessidade de reexecutar o ciclo de treino de alto custo computacional.

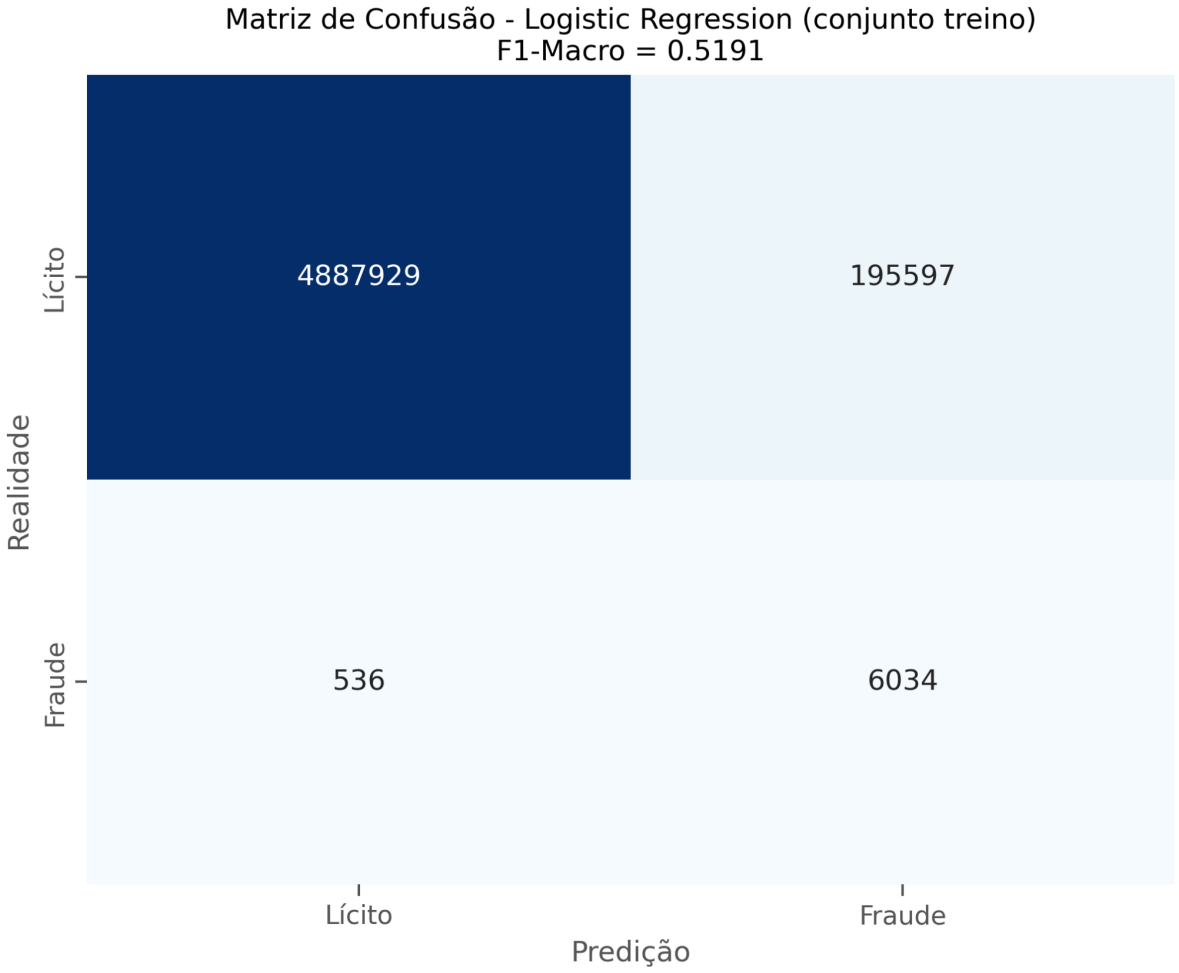
2.3 Pós-processamento (avaliação e figuras)

O script evaluation.py carrega os conjuntos de treino e teste em momentos separados, liberando memória RAM através do garbage collector. Para cada modelo e conjunto de dados, o código gera a matriz de confusão e calcula o respectivo F1-macro. Operando apenas com os dados de teste, o script extrai as dez variáveis de maior influência na decisão do modelo e constrói a curva ROC.

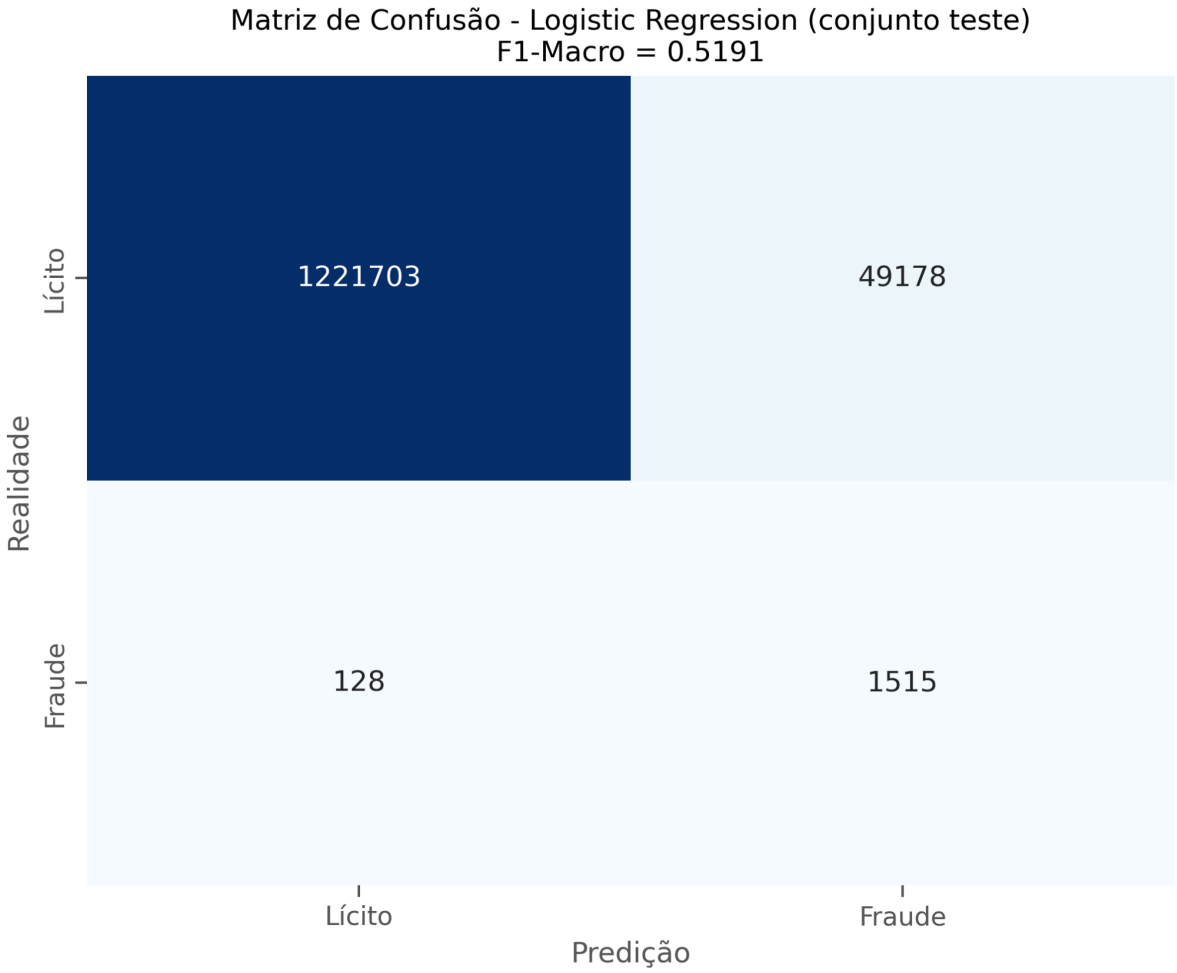
3. Resultados

Os resultados quantificam o desempenho nos dados de teste e comparam o F1-macro aferido no treino e no teste para identificar a ocorrência de overfitting.

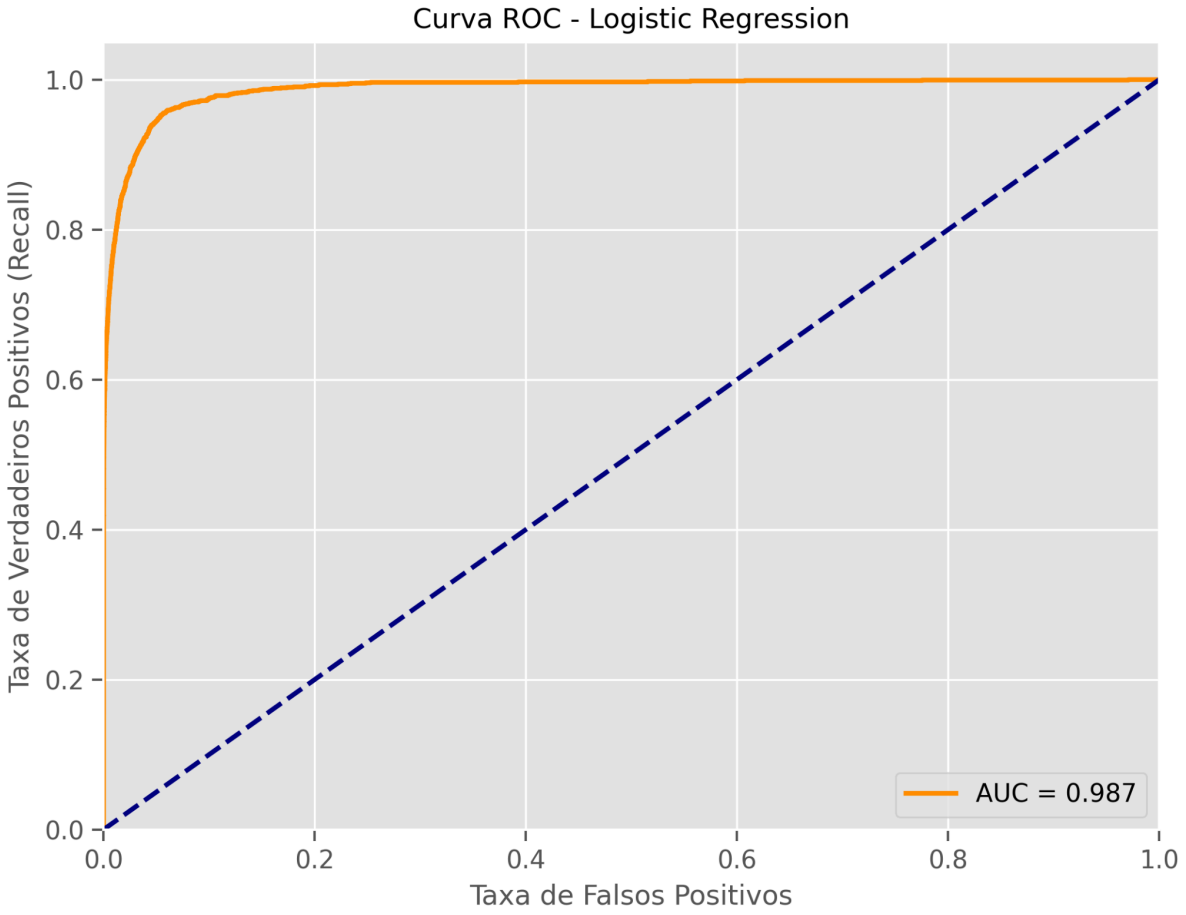
A Regressão Logística obteve o F1-macro de 0.5191 nos dados de treino e 0.5191 nos dados de teste. A matriz de confusão sobre os dados de treino mostra o comportamento do modelo na fase inicial de aprendizado.



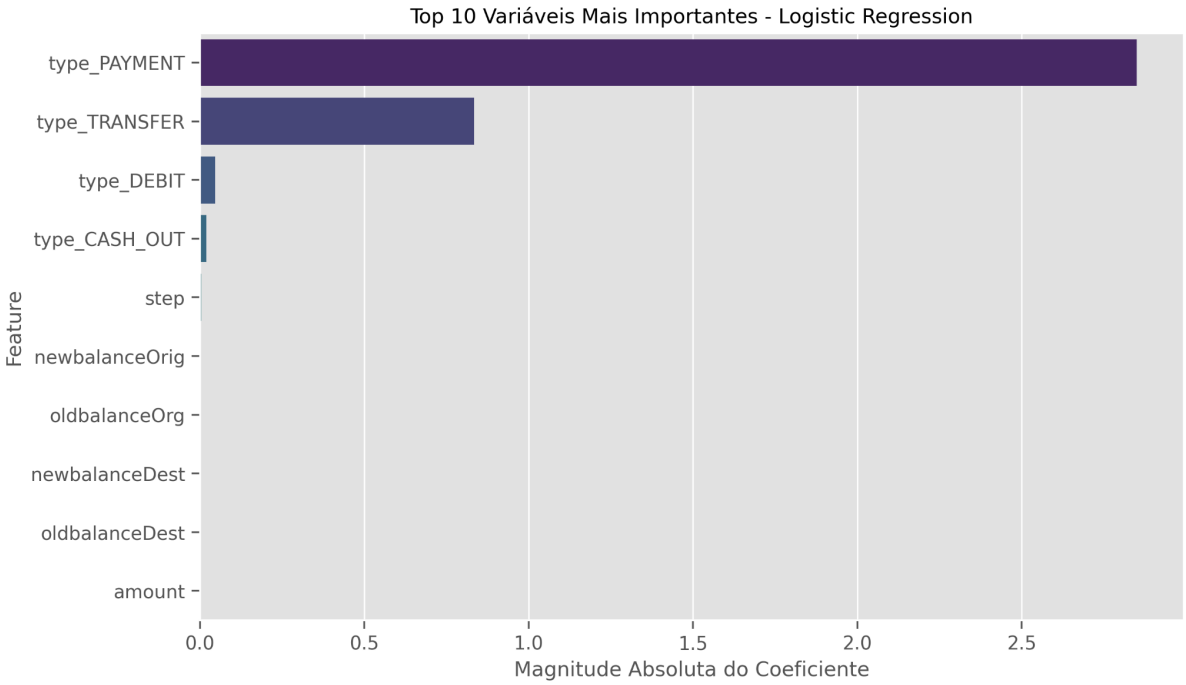
Ao ser aplicada aos dados de teste, a Regressão Logística registrou 49.178 classificações errôneas de transações lícitas (falsos positivos) e detectou 1.515 fraudes verdadeiras.



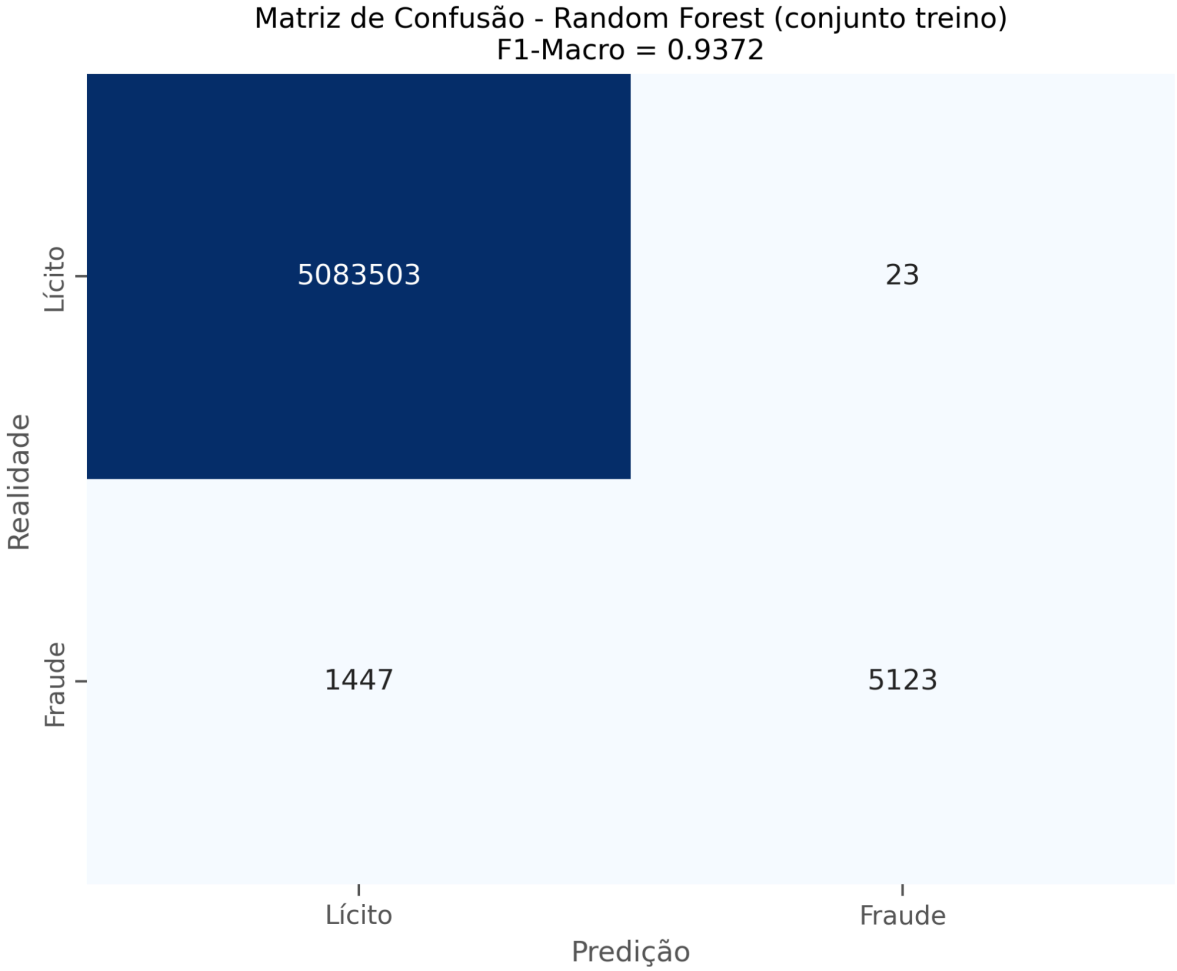
A área sob a curva (AUC) calculada a partir da curva ROC para o modelo de Regressão Logística atingiu a marca de 0.987.



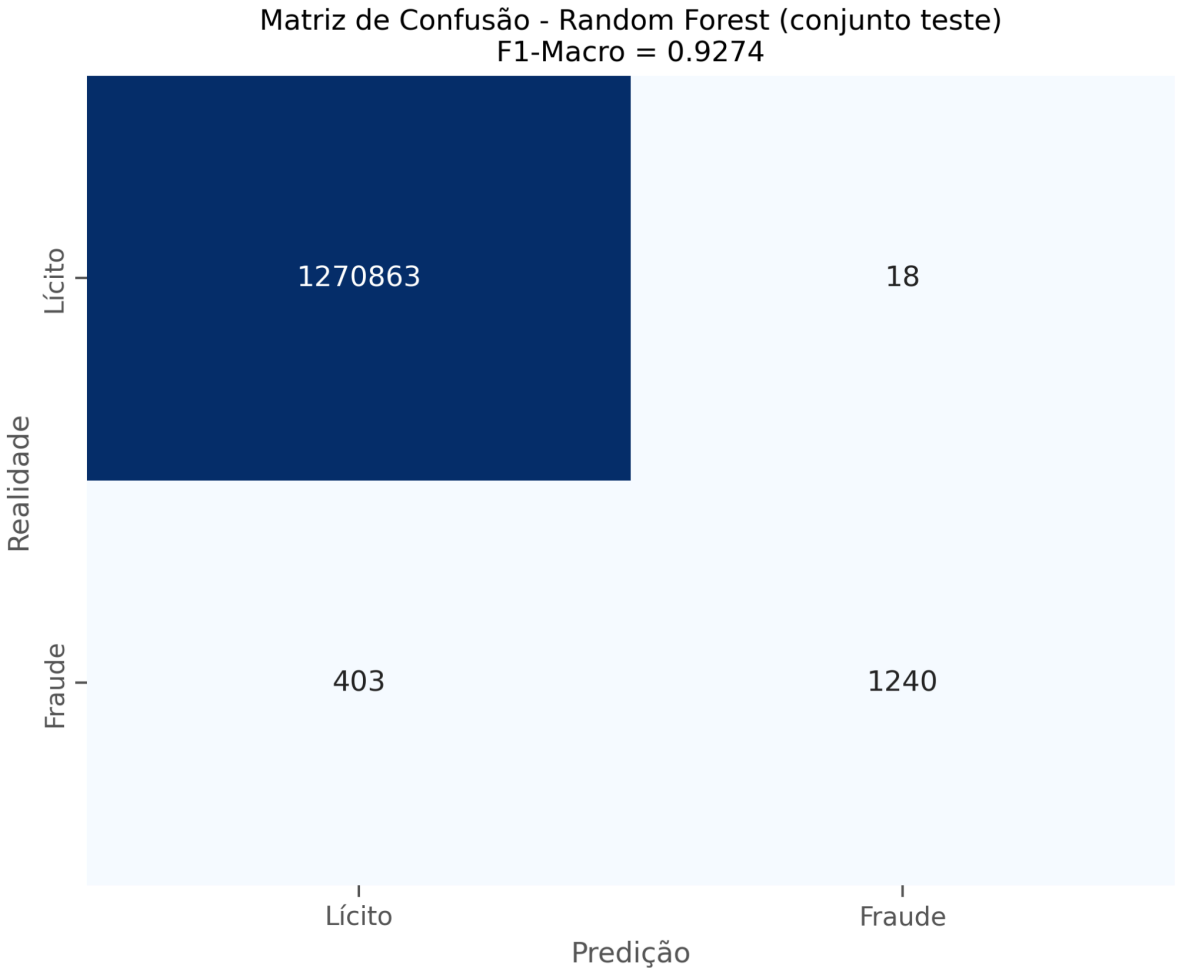
A avaliação da Regressão Logística demonstrou que os coeficientes de maior magnitude absoluta nesse algoritmo linear foram as variáveis categóricas derivadas do tipo de transação: type_PAYMENT, type_TRANSFER, type_DEBIT e type_CASH_OUT.



O modelo Random Forest obteve resultados acentuadamente superiores em relação à Regressão Logística. O F1-macro no conjunto de treino foi de 0.9372.

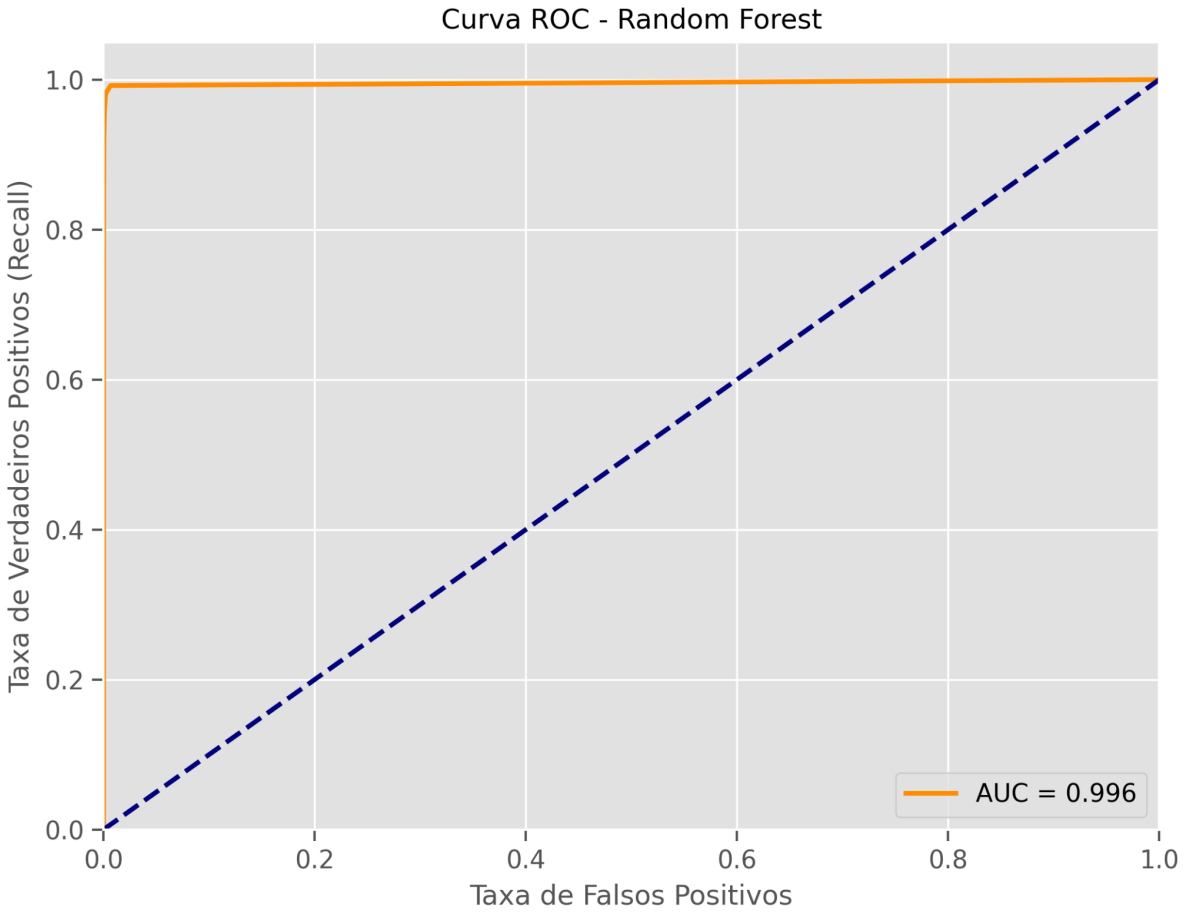


No conjunto de teste, o Random Forest manteve um F1-macro de 0.9274, indicando capacidade sustentada de generalização. O algoritmo detectou 1.240 fraudes reais e reteve os falsos positivos em apenas 18 ocorrências em toda a base de teste.

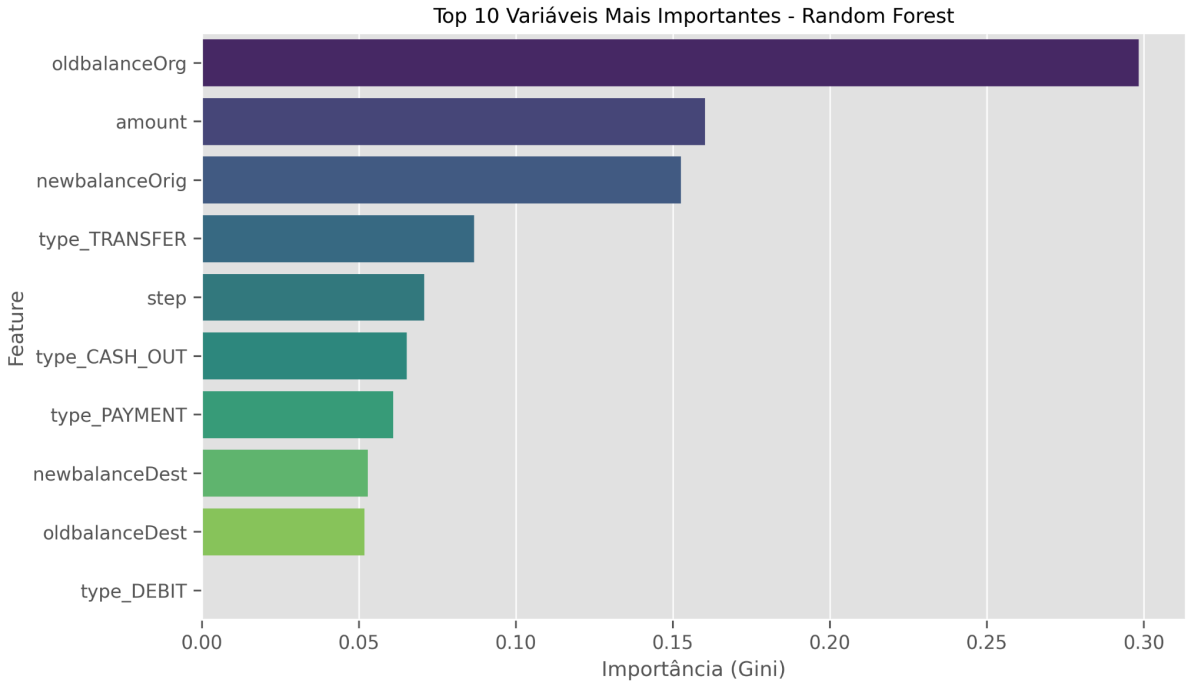


A análise comparativa entre os modelos revela um trade-off analítico na operação financeira. A Regressão Logística apresenta uma área sob a curva (AUC) elevada de 0.987. Contudo, sua aplicação prática resultaria em um alto custo operacional devido ao elevado volume de falsos positivos (49.178 transações lícitas bloqueadas indevidamente). Em contrapartida, o modelo Random Forest demonstrou capacidade superior de filtragem ao restringir os falsos positivos a 18 registros. Essa precisão é necessária para atingir o objetivo de aumentar a confiança do usuário nas plataformas digitais, minimizando os bloqueios desnecessários de usuários legítimos.

A curva ROC gerada a partir das predições do Random Forest apresentou AUC de 0.996.



O algoritmo Random Forest baseou sua classificação levando em consideração múltiplas variáveis de comportamento contínuo. Diferentemente da Regressão Logística, as três variáveis com maior peso para a divisão das árvores de decisão e classificação final das transações foram oldbalanceOrg, amount e newbalanceOrig.



Observa-se uma diferença qualitativa direta na interpretação dos dados por cada algoritmo. A Regressão Logística orientou suas decisões predominantemente pelas variáveis categóricas de tipo de transação. O Random Forest explorou a lógica matemática do problema através das variáveis de saldo e valor da transação, estabelecendo-as como os principais divisores para a classificação correta. O resultado indica que o modelo de florestas aleatórias foi capaz de captar os padrões complexos não lineares da base de dados, como a detecção de contas sendo esvaziadas imediatamente, o que justifica o desempenho superior aferido na métrica F1-macro.

4. Conclusão

O projeto implementou um pipeline completo de mineração de dados para detecção de fraudes em pagamentos online. Os algoritmos Random Forest e Regressão Logística foram validados com 3 folds e avaliados primariamente pela métrica F1-macro. O ajuste dos tipos numéricos e a limitação direta do processamento paralelo permitiram a execução do sistema em um ambiente restrito a 32 GB de memória RAM. O modelo Random Forest reduziu a taxa de falsos positivos de forma mais eficiente do que o modelo linear de Regressão Logística, identificando fraudes através do comportamento do saldo bancário. Possibilidades de trabalhos futuros incluem a sintonia fina avançada de hiperparâmetros, novos testes comparativos com o algoritmo XGBoost e o uso integrado de técnicas sintéticas de aumento de dados em cenários desbalanceados.

Referências

KAGGLE. **Online Payment Fraud Detection**. Disponível em: https://www.kaggle.com/datasets/jainilcoder/online-payment-fraud-detection. Acesso em: 8 mar. 2026.

PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.